

Skill week – 07

WEEK-7:

Task 1: Process Synchronization and Child Process Monitoring Using waitpid().

Task 2: To Retrieve PATH Variable, Parse Search Directories, Locate Executables, Verify Permissions, Handle Missing Commands, Test Command Resolution.

nano task1_waitpid.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {

    pid_t pid;
    int status;

    printf("Parent process started.\n");
    printf("Parent PID: %d\n\n", getpid());

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    /* Child process */
    if (pid == 0) {

        printf("Child process started.\n");
        printf("Child PID: %d\n", getpid());
        printf("Child working...\n\n");

        sleep(3);

        printf("Child process finished.\n");

        exit(10);
    }
```

```

/* Parent process */
else {

    printf("Parent created child with PID: %d\n", pid);
    printf("Parent is waiting for child...\n\n");

    /*
     * Wait specifically for the child process.
     */
    if (waitpid(pid, &status, 0) == -1) {
        perror("waitpid failed");
        return 1;
    }

    printf("\nParent detected that child has finished.\n");

    if (WIFEXITED(status)) {
        printf("Child exited normally.\n");
        printf("Child exit status: %d\n",
            WEXITSTATUS(status));
    }

    printf("Parent process finished.\n");
}

return 0;
}

```

```

bharath@DESKTOP-6D34C2R:~/skill/week7$ nano task1_waitpid.c
bharath@DESKTOP-6D34C2R:~/skill/week7$ gcc task1_waitpid.c -o task1
bharath@DESKTOP-6D34C2R:~/skill/week7$ gcc task1_waitpid.c -o task1
bharath@DESKTOP-6D34C2R:~/skill/week7$ ./task1
Parent process started.
Parent PID: 527

Parent created child with PID: 528
Parent is waiting for child...

Child process started.
Child PID: 528
Child working...

Child process finished.

Parent detected that child has finished.
Child exited normally.
Child exit status: 10
Parent process finished.
bharath@DESKTOP-6D34C2R:~/skill/week7$ |

```

nano task2_path.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <limits.h>

#define MAX_PATH_LENGTH 4096

/*
 * Search for a command in the directories
 * listed in the PATH environment variable.
 */
void find_command(const char *command) {

    char *path;
    char *path_copy;
    char *directory;

    char full_path[PATH_MAX];

    /* Get PATH environment variable */
    path = getenv("PATH");

    if (path == NULL) {
        printf("PATH variable not found.\n");
        return;
    }

    printf("\nPATH variable:\n");
    printf("%s\n\n", path);

    /* Make a copy because strtok modifies the string */
    path_copy = strdup(path);

    if (path_copy == NULL) {
        perror("strdup failed");
        return;
    }

    /* Split PATH using ':' */
    directory = strtok(path_copy, ":");
```

```

while (directory != NULL) {

    /*
    * Construct:
    *
    * directory + "/" + command
    */
    snprintf(full_path,
             sizeof(full_path),
             "%s/%s",
             directory,
             command);

    printf("Checking: %s\n", full_path);

    /*
    * Check whether file exists and
    * has execute permission.
    */
    if (access(full_path, X_OK) == 0) {

        printf("\nFOUND executable!\n");
        printf("Command : %s\n", command);
        printf("Path   : %s\n", full_path);

        free(path_copy);
        return;
    }

    directory = strtok(NULL, ":");
}

printf("\nCommand not found: %s\n", command);

free(path_copy);
}

int main(int argc, char *argv[]) {

    printf("WEEK 7 - TASK 2\n");
    printf("PATH Executable Resolution\n");
    printf("=====\n");

    /*
    * If the user provides a command:
    *
    * ./task2 ls

```

